

UConn · School of Business

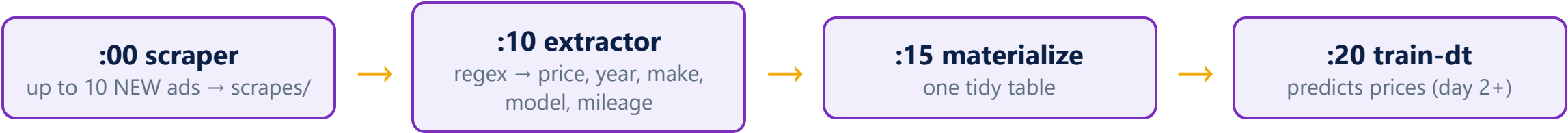
Lab 3 — Scraper Detective

Does your pipeline see what *you* see?

OPIM 5512 · Module 3 · Solo · Colab · your own Google Cloud bucket

Tonight you grade your robot — then fix it with one regular expression.

Your robot has been busy. Is it any good?



Part	Time	You do
1 · Get it running	30 min	5 green checks, then Force run scraper → extractor → materialize
2 · Search and see	25 min	your bucket in Colab · search your cars · open 3 live ads
3 · Grade the robot	30 min	a truth table for 5 cars → accuracy per field
4 · Fix it with regex	30 min	one line: a better make/model regex → re-grade

Solo tonight. Everyone's bucket is a different city and seller type, so everyone's answers are a little different.

Get it running (if it isn't yet)

- 1 • Final GCP Guide F26 (HuskyCT, Week 3.1), Steps 0–10 → **5 green checks** in your fork's Actions tab
- 2 • Cloud Scheduler: **tick the checkbox first**, then **Force run** — one at a time, wait for **Success**:
① `craigslist-scraper-hourly` ② `extractor-per-listing-hourly` ③ `materialize-master-hourly`
- 3 • Cloud Storage → your bucket: `scrapes/` and `structured/` are there
- 4 • Already done? Skip to Part 2 and help a neighbor.

Stuck? Ask Claude first: paste the step, the command, and the FULL error. Then raise your hand.

A map of your bucket

```
gs://your-project-id/
├ scrapes/
│   └ 20260928155246/           # one folder per scraper run (UTC)
│       ├── eH4UEPtCwKz2Vox1upEYCw.txt # the raw ad text
│       └ index.csv              # which URL each file came from
├ state/seen_post_ids.txt       # how it never grabs a car twice
└ structured/
    ├── run_id=20260928155246/jsonl/ # the regex's answers, per ad
    ├── datasets/listings_master.csv  # ← tonight's table
    └ preds/preds_master.csv         # predictions (once you have 2 days)
```

Raw → structured → one table. That's ETL. Tonight you check the T.

Search and see

IN YOUR NOTEBOOK

```
SEARCH_WORD = "toyota" # try sunroof, 4wd...

for i in range(len(cars)):
    row = cars.iloc[i]
    text = read_ad_text(row["source_txt"])
    if SEARCH_WORD.lower() in text.lower():
        matches.append(row)

3 of 10 ads mention 'toyota'
```

THEN OPEN THE REAL AD NEXT TO ITS RAW TEXT

```
1980 Lincoln Continental Town Coupe
$4,450
image 1 of 24
1980
lincoln continental
condition:
good
odometer:
47,939
```

Why search the raw text and not the make column? Look at the make column first. You'll see.

Grade the robot: a truth table

Open 5 ads. Write down what each ad **really** says. Compare it to what the regex extracted:

post_id	price	year	make	model	mileage
eH4UEPt...	4450 ✓	1980 ✓	Contact ✗	Information ✗	47939 ✓
5H9PrAV...	7000 ✓	2017 ✓	Contact ✗	Information ✗	188500 ✓
... 3 more of yours					
accuracy	?	?	0%?	0%?	?

Your prices might not be perfect either. What happens when an ad says "\$588/month" before the real price? Find one.

THE MYSTERY

Why does every car say "Contact Information"?

THE EXTRACTOR'S RULE (MODULE 3.2)

```
MAKE_MODEL_RE = re.compile(
    r"\b([A-Z][a-z]+)\s+([A-Z][A-Za-z0-9]+)")

# in words: the FIRST place where one
# Capitalized word is followed by another
```

WHAT THE PAGE SAYS, TOP TO BOTTOM

```
'Posted'
'2026-09-27 17:35'
'Contact Information:' ← first match!
'print'
'1980 Lincoln Continental Town Coupe'
...
'1980'
'lincoln continental' ← the real answer
```

Regex does exactly what you say, not what you mean. Craigslist changed its page, and "first two capitalized words" became a button label.

Build it one piece at a time

piece	means	piece	means
<code>^</code> <code>\$</code> + <code>re.MULTILINE</code>	start / end of a line	<code>\n</code>	the line break
<code>\d{2}</code>	exactly two digits	<code>\s+</code>	one or more spaces
<code>(?:19 20)</code>	19 or 20, grouped, not captured	<code>(...)</code>	capture group → <code>.group(1)</code>

```
step 1  a line that is only a year      r"^(?:19|20)\d{2}$"      → matches '1980'
step 2  ...followed by the next line    r"^(?:19|20)\d{2}\n"
step 3  capture the first word (make)    your turn
step 4  a space, then capture the model  your turn
```

Test fast at regex101.com (Python flavor). Paste 20 lines of an ad as the test string and watch the match light up.

Fix it with ONE line, then re-grade

YOU WRITE

```
BETTER_MAKE_MODEL_RE = re.compile(
    r"YOUR PATTERN HERE", re.MULTILINE)

better_make_model(text)
# ('lincoln', 'continental')
```

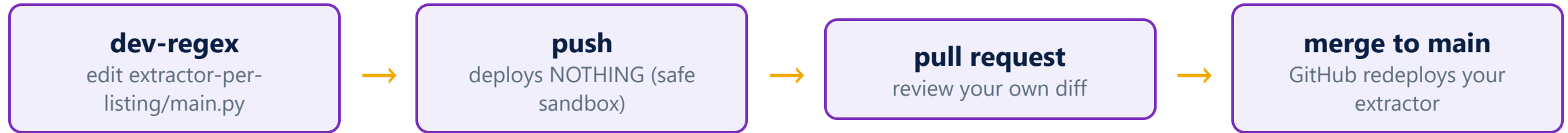
THE NOTEBOOK RE-GRADES YOUR 5 CARS

field	before	after
price	...	...
make	0.0	?
model	0.0	?

Stretch: run it on **all** your cars. How many ads does your pattern miss, and why?

One regular expression. One better dataset. That's what "update the ETL" means in A06.

Ship your regex: **branch** → **PR** → **deploy**



- Only **main** deploys: the workflows run on main, and Google Cloud only trusts main.
- Keep your own analysis in **separate files** (notebooks) so **Sync fork** still works when Dr. Wanik updates the course code.

That's CI/CD: code change → review → automatic deploy. No clicking around in the cloud.

Good advice

- **Colab pop-up:** pick the Gmail you used for **Google Cloud**, not necessarily your UConn account.
- **listings_master.csv not found?** Force run the extractor, then materialize-master; wait for **Success**.
- **Your regex matches nothing?** Print the lines with `repr()`: hidden spaces and line breaks show up.
- **Only 10 cars?** That's one scrape. It adds up to 10 **new** cars every hour; leave it running.
- **Force run the scraper once**, then the extractor. Two scrapes are fine now, but one is all you need.

DEFINITION OF DONE

Submit Lab 3 on HuskyCT

- ☐ Part 2: search results + 2–3 sentences on what the raw text keeps and loses
- ☐ Part 3: your **truth table** for 5 cars + the **before** accuracy table
- ☐ Part 4: your **BETTER_MAKE_MODEL_RE** line + the **before vs after** table
- ☐ Part 5: three short answers
- ☐ File → Download → **.ipynb** → **Submit Lab 3** (due **Thu Oct 8, 11:59 PM**)

Leave the robot running. Next: A06 (update the ETL), then A07, where Gemini reads the ads instead of a regex.